Variables and Operators

Lecture 1 · Learning Python at the University of Glasgow

Khiem Nguyen

9 October 2026

Contents

1 Literals: values written directly in code **3**

2 Variables **4**
2.1 Why do we need variables? . . . 4
2.2 Variables in a nutshell . . . 5
2.3 Naming rules and conventions . . . 5
2.4 Creating and updating variables . . . 6

3 Basic data types **7**
3.1 Converting between types . . . 8
3.2 Everything is an object . . . 9

4 Operators **10**
4.1 Types of operators . . . 10
4.2 Arithmetic operators . . . 11
4.3 Comparison operators . . . 11
4.4 Logical operators . . . 12
4.5 Membership operators . . . 13
4.6 Identity operators . . . 13
4.7 Assignment operators . . . 14
4.8 Operator precedence and associativity . . . 15

5 Strings **16**
5.1 Creating strings . . . 16
5.2 Operations on strings . . . 17
5.3 String methods . . . 18

6 Output with `print()` **19**
6.1 The basics . . . 19
6.2 Formatting output with `str.format()` . . . 20
6.3 f-strings: the modern way . . . 21

7 A first look at modules **21**

8 Input with `input()` 22
8.1 Example: the roots of a quadratic equation 23

Summary 24

Exercises 24
Exercise 1: Just a start 24
Exercise 2: NumPy and its documentation 24
Exercise 3: Mathematical functions 24
Exercise 4: Strings and their operations 25
Exercise 5: Fancy output with string formatting 26
Exercise 6: Projectile motion 26

Further reading 27

Learning objectives

By the end of this lecture, you should be able to:

- distinguish between *literals* (fixed values) and *variables* (names that refer to values);
- create, update and delete variables, and choose good variable names;
- recognise Python's basic data types and convert between them;
- use arithmetic, comparison, logical, membership, identity and assignment operators;
- create and manipulate strings, and display neatly formatted output with `print()`;
- import functions from modules such as `math` and `numpy`;
- read input from the keyboard with `input()`.

1 Literals: values written directly in code

A **literal** is a fixed value written directly in the source code, such as `999`, `3.14`, `True` or `"Hello World"`. Literals are constants — literally!

- Python works out by itself whether a literal is an integer, a floating-point number, a Boolean value or a string, and organises the memory storage accordingly.
- String literals are written between single quotes (`'...'`) or double quotes (`"..."`); triple quotes (`"""..."""`) allow a string to span several lines.

Comments

Everything that follows a hash sign `#` on a line is a **comment**. Comments are ignored by Python; they exist only to help humans read and understand the code. Use them generously!

A remark on the output of code cells

In the next few cells, we deliberately put a single literal in each cell. This is not good practice in general, and we will not do it again in this course. We do it here only to show that, in Jupyter, the value of the **last** expression in a cell is displayed automatically, without any help from `print()`. If several literals are written in the same cell, only the last one is shown. To display every value, pass each of them to the built-in function `print()`.

```
# An integer literal
999
```

```
999
```

```
# The keyword True is a Boolean literal. Its counterpart is False.
True
```

```
True
```

```
# A string literal: text enclosed in double quotes "..."
"Hello World"
```

```
'Hello World'
```

```
999            # evaluated, but not displayed
True           # evaluated, but not displayed
"Hello World"  # only the value of the last expression is displayed
```

```
'Hello World'
```

```
print(999)            # print the integer literal
print(True)           # print the Boolean value
print("Hello World")  # print the string
```

```
999
True
Hello World
```

2 Variables

2.1 Why do we need variables?

Literals that we never use again are useless. Solving complicated problems with a computer is only possible if we can **store** data and intermediate results, and **retrieve** them later. Without a way of storing values, we could not even use a pocket calculator! During a computation, data are kept in the computer's memory, usually the *random-access memory* (RAM). Every programming language therefore needs a mechanism for saving values in memory and accessing them whenever they are needed. This is what **variables** are for.

You can think of **a variable as a name that refers to a value stored somewhere in memory**. Through this name, we can read the value, replace it with a new one, or delete the name altogether when we no longer need it.

> **You already know variables**
>
> You have been working with variables since secondary school. When you solve the linear equation $ax + b = 0$ for the unknown x, the letters a and b hold actual numbers, and the solution is "stored" in x. Similarly, when you solve the quadratic equation $a_2x^2 + a_1x + a_0 = 0$ for x, the coefficients a_0, a_1 and a_2 hold actual values.

2.2 Variables in a nutshell

- A variable is a **name** (an alias) that refers to a value stored in memory.
- The actual location in physical memory is chosen by Python, not by you, and it may change from one run to the next.
- In CPython (the standard implementation of Python), the built-in function `id()` returns a number that identifies the object a name refers to — in practice, its memory address.

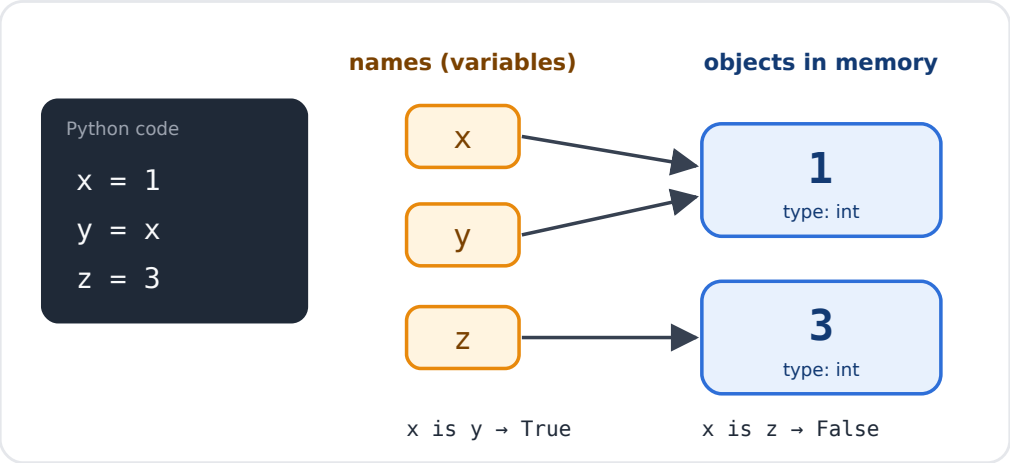


Figure 1: Variables are names that refer to objects. After `y = x`, both `x` and `y` refer to the *same* object, whereas `z` refers to a different one.

2.3 Naming rules and conventions

Rules (enforced by Python):

- Names may contain letters, digits and the underscore `_`.
- The first character must be a letter or an underscore — never a digit.
- Names are case-sensitive: `age`, `Age` and `AGE` are three different variables.
- Python **keywords**, such as `if`, `for`, `while`, `True` and `def`, cannot be used as names. (Run `import keyword; print(keyword.kwlist)` to see the full list.)

Good practice (not enforced, but strongly recommended):

- Choose meaningful names: `my_age`, `initial_speed`, `profit` rather than `a`, `b`, `c`.
- Avoid names that are easily confused, such as `l` (lower-case L), `O` (upper-case o) and `I` (upper-case i) — unless your aim is to confuse!
- Avoid the names of built-in functions, such as `print`, `sum`, `max`, `min`, `list` and `str`. Python allows it, but the original function is then hidden ("shadowed"), and you can no longer use it.

There are two popular conventions for names made of several words:

1. **Camel case**: `myAge`, `initialSpeed`, `myProfit`
2. **Snake case** (the underscore convention): `my_age`, `initial_speed`, `my_profit`

Whichever you choose, be consistent. Python's official style guide, PEP 8, recommends snake case for variables and functions, and capitalised words (e.g. `ElectricCar`) for class names, which we meet in Lecture 5. **Throughout this course, we use snake case.** Some older languages restricted the length of names, which encouraged short and cryptic names; modern languages have no such restriction, and underscores make long names much easier to read.

> **Long names cost nothing**
>
> There is no need to save keystrokes on variable names. Modern editors and IDEs (Jupyter, VS Code, PyCharm, Spyder, …) complete names for you: type the first few letters and press **Tab**.

2.4 Creating and updating variables

Creating a variable. A variable is created by an **assignment**: the value on the right-hand side (RHS) of the equals sign `=` is computed first, and the result is then bound to the name on the left-hand side (LHS). The RHS may be a literal, another variable, or a complex expression involving several variables.

For example, in the statement `i = 7`, the RHS is evaluated and the result is assigned to `i`. We may read this as "`i` becomes equal to 7", or "`i` holds the value 7". In the statement

```
result = (result_1 + result_2 / result_3) * result_4
```

Python first evaluates the RHS using the values currently held by `result_1`, `result_2`, `result_3` and `result_4`, and then assigns the outcome to `result`. In both cases, Python brings the variable into existence at the moment it is first assigned; there is no need to declare it in advance.

Updating a variable. To update a variable, we simply assign a new value to the existing name.

```
# 'Albert' is a literal; first_name is a variable (single quotes)
first_name = 'Albert'
last_name = "Einstein"  # double quotes work just as well
```

```
# evaluate the RHS, then assign it to the LHS
full_name = first_name + ' ' + last_name
```

```
# The variables keep their values, so we can use them at any time.
# print() writes whatever we give it to the output area.
print("Hello world")   # apply a function to a literal
print(first_name)      # apply a function to a variable
print(last_name)
print(full_name)
```

```
Hello world
Albert
Einstein
Albert Einstein
```

```
# Some more (rather useless) code, just for fun
my_look_age = 25                    # my_look_age becomes equal to 25
my_real_age = 35                    # my_real_age becomes equal to 35
ratio = my_look_age / my_real_age   # division, just like at school
print(ratio)
```

```
0.7142857142857143
```

```
# Updating a variable: the right-hand side uses the OLD value
speed = 10
print(speed)
speed = speed + 5    # read as: "speed becomes the old speed plus 5"
print(speed)
```

```
10
15
```

3 Basic data types

Every value in Python has a **type**, which determines what the value represents and which operations are allowed on it. Python works out the type automatically from the value; you never need to declare it. The most common basic types are:

Type	Meaning	Examples
`int`	whole numbers (of any size)	`7`, `-42`, `10**100`
`float`	floating-point (real) numbers	`3.14`, `-0.5`, `2.0`, `1.5e-3`
`complex`	complex numbers	`3 + 4j`
`bool`	Boolean (truth) values	`True`, `False`
`str`	text (strings)	`'spam'`, `"Hello World"`
`NoneType`	"no value at all"	`None`

The built-in function `type()` tells us the type of any value.

```
print(type(7))
print(type(3.14))
print(type(3 + 4j))
print(type(True))
print(type("Hello"))
print(type(None))
```

```
<class 'int'>
<class 'float'>
<class 'complex'>
<class 'bool'>
<class 'str'>
<class 'NoneType'>
```

3.1 Converting between types

The names of the types can also be used as functions that convert a value into that type. This is particularly important when we read input from the keyboard, which always arrives as a string (see the section on input below).

```
print(int("42") + 1)      # string -> integer
print(float("3.5") * 2)   # string -> float
print(str(2024) + "!")    # integer -> string
# float -> integer: the decimal part is cut off (not rounded!)
print(int(3.99))
print(round(3.99))        # use round() if you want rounding
print(bool(0), bool(5))   # 0 converts to False; any other number converts
 ↪  to True
```

```
43
7.0
```

```
2024!
3
4
False True
```

A conversion that does not make sense raises an error:

```
int("3.5")   # ValueError: "3.5" is not the text of an integer
```

```
ValueError: invalid literal for int() with base 10: '3.5'
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[229], line 1
----> 1 int("3.5")   # ValueError: "3.5" is not the text of an integer
ValueError: invalid literal for int() with base 10: '3.5'
```

3.2 Everything is an object

We will study **classes** and **objects** properly in Lecture 5. Roughly speaking, an *object* is an entity that has its own data, features and actions, and a *class* is the template from which such objects are made.

For example, "Mary" is a particular object of the template "Student at the University of Glasgow". Mary has a name, a GUID and a list of registered courses and, like every other student, she has to enrol on courses and pass exams in order to graduate. Likewise, "Iron Man" is a particular object of the template "Superhero": he has a real name and special powers, and he can do fancy things, just like every other superhero — although each hero has their own name, powers and tricks.

In Python:

- Every value — integers, floating-point numbers, strings, and so on — is actually an object, and a variable is simply a name bound to an object.
- Objects carry useful information with them. For instance, a string knows its own length, so Python never has to count its characters again.
- Because a variable is only a name, we can remove the name with `del`. Afterwards, the name is undefined.

```
x = 42
print(x)
del x       # remove the name x
print(x)    # NameError: name 'x' is not defined
```

```
42
```

```
NameError: name 'x' is not defined
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[230], line 4
      1 x = 42
      2 print(x)
      3 del x       # remove the name x
----> 4 print(x)    # NameError: name 'x' is not defined
NameError: name 'x' is not defined
```

4 Operators

In simple terms, **operators** are symbols that act on values or variables to produce new values — mostly maths, just like at school. Addition, subtraction, multiplication and division are all operators acting on two numbers (integers or floating-point numbers). In more technical language, **operators** are constructs defined within a programming language that perform operations on values and variables, which are called **operands**.

> ! Take-home message
>
> - Every program is a sequence of modifications of variables — that is what an algorithm is about.
> - These modifications happen through operators, e.g. plus `+`, minus `-`, multiply `*`, divide `/`, etc.
> - Python's built-in libraries provide more advanced operators, shortcuts and functions. We will learn them over time.

4.1 Types of operators

Python supports the following types of operators:

- arithmetic operators;
- comparison (relational) operators;
- logical operators;
- membership operators;
- identity operators;
- assignment operators;
- bitwise operators.

We study all of them except the bitwise operators, which act on the individual bits of integers and are rarely needed in this course.

4.2 Arithmetic operators

Assuming `a = 10` and `b = 20`:

Operator	Description	Example
`+`	addition	`a + b` gives `30`
`-`	subtraction	`a - b` gives `-10`
`*`	multiplication	`a * b` gives `200`
`/`	division	`a / b` gives `0.5`
`%`	modulus (remainder of a division)	`a % b` gives `10`
`**`	exponentiation (power)	`2 ** 4` gives $2^4 = 16$
`//`	floor division (round *down* the quotient)	`9 // 2` gives `4`; `-11 // 3` gives `-4`

Note that the division `/` always returns a `float`, even when the result is a whole number.

```
a, b, c, d, e, f, g = 5, 1, 2, 3, 4, 7, 8
# This is a short way to assign several values to several variables at
 ↪  once:
# a = 5, b = 1, c = 2, d = 3, e = 4, f = 7, g = 8
print(g // f)
print(c ** d)
print(a + b * c * d % e - f / g)
# the same expression, with brackets showing the order
print(a + (b * c * d) % e - (f / g))
print(4 / 2)  # / always returns a float
```

```
1
8
6.125
6.125
2.0
```

4.3 Comparison operators

Comparison operators compare two values and return a Boolean value: `True` or `False`.

Operator	Description
<	less than
<=	less than or equal to
>	greater than
>=	greater than or equal to

Operator	Description
`==`	equal to
`!=`	**not** equal to

> ⚠ = is not ==
>
> A single equals sign = *assigns* a value to a variable; a double equals sign == *compares* two values.

```
a, b, c, d = 1, 2, 3, 3
print("a < b  :", a < b)
print("a > b  :", a > b)
print("c <= d :", c <= d)
print("c >= d :", c >= d)
print("a == b :", a == b)
print("a != b :", a != b)
```

```
a < b  : True
a > b  : False
c <= d : True
c >= d : True
a == b : False
a != b : True
```

4.4 Logical operators

Boolean values can be combined with logical operators. We rarely store Boolean values explicitly in variables; they usually come from comparisons, as in the last example of the next cell.

Operator	Description
`not`	negation: `not x` is `True` if `x` is `False`
`and`	`x and y` is `True` only if **both** `x` and `y` are `True`
`or`	`x or y` is `True` if **at least one** of them is `True`

```
print("True and False =", True and False)
print("False or True  =", False or True)
print("not False      =", not False)
print("not True       =", not True)

temperature = 18
print("Pleasant weather:", temperature > 15 and temperature < 25)
```

```
True and False = False
False or True  = True
not False      = True
not True       = False
Pleasant weather: True
```

4.5 Membership operators

Membership operators test whether a value occurs in a sequence. We use them with strings for now, and with lists, tuples and dictionaries in Lecture 3.

Operator	Description
`in`	`True` if the value is found in the sequence
`not in`	`True` if the value is **not** found in the sequence

```
word = "engineering"
print("gin" in word)
print("z" in word)
print("z" not in word)
```

```
True
False
True
```

4.6 Identity operators

Identity operators test whether two names refer to the **same object** (recall the figure in the section *Variables in a nutshell*).

Operator	Description
`is`	`True` if both refer to the same object
`is not`	`True` if they refer to **different** objects

```
x = 1
y = x
z = 3
# y refers to the same integer object as x, so their IDs are identical.
# z refers to a different object, so its ID is different.
print("id(x) =", id(x))
print("id(y) =", id(y))
```

```
print("id(z) =", id(z))
print("x is y     :", x is y)
print("x is not y :", x is not y)
print("x is z     :", x is z)
print("x is not z :", x is not z)
```

```
id(x) = 108146729163472
id(y) = 108146729163472
id(z) = 108146729163536
x is y     : True
x is not y : False
x is z     : False
x is not z : True
```

⚠ `is` versus `==`

`==` checks whether two values are *equal*; `is` checks whether two names refer to the *same object*. Two objects can be equal without being the same object, as the next cell shows (it uses lists, which we study in Lecture 3). Use `==` to compare values, and keep `is` for special cases such as `x is None`.

```
p = [1, 2, 3]
q = [1, 2, 3]
print(p == q)   # True: the two lists have equal contents
print(p is q)   # False: they are two different objects
```

```
True
False
```

4.7 Assignment operators

- The equals sign `=` is the **assignment operator**: it assigns the value of the right-hand side to the variable on the left-hand side. So `x = 3 + 5` is fine, but `8 = 3 + 5` is a syntax error, because nothing can be assigned to the literal `8`.
- Statements of the form `n = n + 1` are so common that Python offers a shortcut: `n += 1`. Note that the equals sign comes *after* the arithmetic operator. The same shortcut exists for all the arithmetic operators.

Operator	Example	Equivalent to
`=`	`x = 3`	x becomes equal to 3
`+=`	`x += 3`	`x = x + 3`
`-=`	`x -= 3`	`x = x - 3`

Operator	Example	Equivalent to
`*=`	`x *= 3`	`x = x * 3`
`/=`	`x /= 3`	`x = x / 3`
`%=`	`x %= 3`	`x = x % 3`
`//=`	`x //= 3`	`x = x // 3`
`**=`	`x **= 3`	`x = x ** 3`

```
n = 10
n += 1      # n = n + 1
print(n)
n *= 3      # n = n * 3
print(n)
n //= 4     # n = n // 4
print(n)
```

```
11
33
8
```

4.8 Operator precedence and associativity

- **Precedence** works as in ordinary mathematics: `**` binds more tightly than `*` and `/`, which in turn bind more tightly than `+` and `-`.
- When operators have the same precedence, **associativity** decides the order of evaluation. Most operators are evaluated from **left to right**; for example, `10 - 4 - 3` means `(10 - 4) - 3 = 3`.
- Exponentiation `**` is the exception: it is evaluated from **right to left**, so `2 ** 3 ** 2` means `2 ** (3 ** 2) = 512`.

Always use brackets if you are even vaguely unsure. They cost nothing and make your intention clear. The complete precedence table is in the official Python documentation.

```
print(10 - 4 - 3)      # left to right: (10 - 4) - 3
print(2 ** 3 ** 2)     # right to left: 2 ** (3 ** 2)
print((2 ** 3) ** 2)   # brackets change the order
# careful: ** binds more tightly than the minus sign, so this is -(2 ** 2)
print(-2 ** 2)
```

```
3
512
64
-4
```

5 Strings

Strings are among the most natural data types we use every day, alongside integers and floating-point numbers. A string is a sequence of characters, so it can hold a single letter, a word or even a whole paragraph.

5.1 Creating strings

Strings can be enclosed in single quotes (`'...'`) or double quotes (`"..."`) with the same result. The backslash `\` is used to **escape** characters that would otherwise have a special meaning: to put a single quote inside a string enclosed in single quotes, write `\'`; similarly, `\"` puts a double quote inside a string enclosed in double quotes. The most useful escape sequences are:

Escape sequence	Meaning
`\n`	new line
`\t`	tab
`\'`	single quote
`\"`	double quote
`\\`	a backslash itself

```
string_1 = 'spam eggs'                    # single quotes
string_2 = 'It doesn\'t mean food.'       # use \' to escape the single
 ↪  quote
# Inside double quotes, a single quote needs no escape character.
string_3 = "It doesn't mean food."
# This makes sense: if a string started with a single quote, how could
 ↪  Python
# tell whether the next single quote ends the string or belongs to it?

# The character "\n" starts a new line, so this string is printed on two
 ↪  lines.
string_4 = "This is the first line.\nThis is the second one."
# The character "\t" inserts a tab, i.e. some horizontal space, like the
 ↪  TAB key.
string_5 = "I create a tab \t and you see so much space.\t\t What, not
 ↪  again!"
string_6 = "Compare a single space ' ' with a tab '\t'."

print(string_1)
print(string_2)
```

```
print(string_3)
print(string_4)
print(string_5)
print(string_6)
```

```
spam eggs
It doesn't mean food.
It doesn't mean food.
This is the first line.
This is the second one.
I create a tab    and you see so much space.      What, not again!
Compare a single space ' ' with a tab ' '.
```

Triple quotes create a string that spans several lines:

```
poem = """Roses are red,
violets are blue,
Python is great,
and so are you."""
print(poem)
```

```
Roses are red,
violets are blue,
Python is great,
and so are you.
```

5.2 Operations on strings

- Strings can be concatenated (glued together) with the `+` operator, and repeated with the `*` operator.
- Two or more string *literals* (i.e. text enclosed between quotes) written next to each other are concatenated automatically. This is particularly useful for breaking long strings over several lines.
- The built-in function `len()` returns the number of characters in a string.
- Individual characters are accessed by their position (counting from zero), e.g. `word[0]`. We study indexing in detail with lists in Lecture 3.

```
a = "You"
b = "are"
c = "awesome"
# strings stored in variables, concatenated
print(a + " " + b + " " + c + "!")
```

```
print("I" + " am" + " Coffee " + "Milk.")   # with plus signs
print("I" " am" " Coffee " "Milk.")         # without plus signs: adjacent
 ↪  literals
print(3 * "ha")                             # repetition
print(len("engineering"))                   # number of characters
print("engineering"[0], "engineering"[-1])  # first and last character
```

```
You are awesome!
I am Coffee Milk.
I am Coffee Milk.
hahaha
11
e g
```

> **!** Strings are immutable
>
> Once a string has been created, it cannot be changed. Operations that seem to modify a string actually create a **new** string.

```
greeting = "Hello"
greeting[0] = "J"    # TypeError: strings do not support item assignment
```

```
TypeError: 'str' object does not support item assignment
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[242], line 2
      1 greeting = "Hello"
----> 2 greeting[0] = "J"    # TypeError: strings do not support item assignment
TypeError: 'str' object does not support item assignment
```

5.3 String methods

Every string is an object, so it comes with many useful **methods** — functions attached to the object — called with the dot syntax `my_string.method_name()`. Here are a few of them:

```
s = "  Python for Engineers  "
print(s.upper())                    # all upper case
print(s.lower())                    # all lower case
print(s.strip())                    # remove leading and trailing spaces
print(s.replace("Python", "Maths")) # returns a NEW string
print("Hello".replace("H", "J"))    # the way to "change" a string
```

```
print("12345".isnumeric(), "a1b2".isnumeric())
```

```
  PYTHON FOR ENGINEERS
  python for engineers
Python for Engineers
  Maths for Engineers
Jello
True False
```

6 Output with `print()`

Producing neat output was historically one of the messiest parts of many programming languages; output conventions date from a time when dot-matrix line printers were cool! Fortunately, Python makes it fairly painless. We discuss output first and input afterwards.

6.1 The basics

- `print()` accepts several values separated by commas. By default, it separates them with a single space; the keyword argument `sep` changes the separator.
- `print()` always ends with an invisible newline character `\n`. You can change this with the keyword argument `end`; for example, `print('something', end='')` prints nothing after its argument, so the next output continues on the same line.

```
print("Something")
print("Wrong")                    # a new line starts after "Wrong"

print("--------------------")
print("Something", end="")        # NO new line
print("Else", end=",\n")          # a comma and then a new line after "Else"

print("--------------------")
print("x", "y", "z")              # separated by spaces (the default)
print("x", "y", "z", sep=" | ")   # a custom separator
```

```
Something
Wrong
--------------------
SomethingElse,
--------------------
x y z
x | y | z
```

6.2 Formatting output with str.format()

Because strings are objects, they come with the method format(), which is used to produce fancy output. Best explained by an example:

```
print("My text {} {} {}.".format(arg1, arg2, arg3))
```

prints My text and then substitutes the values of the three arguments arg1, arg2 and arg3 into the three placeholders {}, in order of appearance. For example, if arg1 = 'is', arg2 = 'very' and arg3 = 'useful', the output is My text is very useful.

Each placeholder can contain a specification of the form {<select>:<format>}. Here, <select> is the position of the argument in the argument list (counting from zero), so the following two lines are identical:

```
print("a = {}, b = {}".format(arg1, arg2))
print("a = {0}, b = {1}".format(arg1, arg2))
```

The <format> part, after the colon, controls the alignment, width, precision and so on. The most useful specifications are:

Specification	Meaning	Example result for 3.14159 / 'hi'
{:<10}	left-align in a field 10 characters wide	'hi '
{:>10}	right-align in a field 10 characters wide	' hi'
{:^10}	centre in a field 10 characters wide	' hi '
{:*^10}	centre, filling the space with *	'****hi****'
{:.3f}	fixed-point notation with 3 decimal places	'3.142'
{:10.3f}	width 10, 3 decimal places	' 3.142'
{:.2e}	scientific notation with 2 decimal places	'3.14e+00'
{:+f}	always show the sign	'+3.141590'

The following examples are self-explanatory:

```
print("{:<40}".format('left aligned'))              # < : aligned to the
 ↪ left
print("{:>40}".format('right aligned'))             # > : aligned to the
 ↪ right
print("{:^40}".format('centred'))                   # ^ : centred
print("{:*^40}".format('centred with asterisks'))   # fill with * and then
 ↪ centre
```

```
left aligned
                            right aligned
                centred
*********centred with asterisks*********
```

```
print("{0:+f}; {1:+f}".format(3.14, -3.14))   # always show the sign
print("{0: f}; {1: f}".format(3.14, -3.14))   # a space instead of the plus
 ↪ sign
print("{0:-f}; {1:-f}".format(3.14, -3.14))   # only the minus sign (the
 ↪ default)
print("{0:.2f} and {0:.2e}".format(1234.5678))  # the same argument used
 ↪ twice
```

```
+3.140000; -3.140000
 3.140000; -3.140000
3.140000; -3.140000
1234.57 and 1.23e+03
```

6.3 f-strings: the modern way

Since Python 3.6, there has been an even more convenient way of formatting: **formatted string literals**, or **f-strings**. Put the letter `f` before the opening quote and write variables — or any expression — directly inside the braces. The format specifications after the colon are exactly the same as above. f-strings are the recommended style for new code, but you will meet both styles in these notes and elsewhere.

```
name = "Ada"
speed = 12.3456
print(f"{name} cycles at {speed:.2f} m/s.")
print(f"{'centred':*^20}")
print(f"2 + 3 = {2 + 3}")
```

```
Ada cycles at 12.35 m/s.
******centred*******
2 + 3 = 5
```

7 A first look at modules

Python comes with a large **standard library** organised into **modules**, and there are thousands of third-party packages such as NumPy. To use the functions in a module, we first `import` it. There are three common forms of the import statement:

```
import math                  # import the whole module ...
print(math.sqrt(16))         # ... and use its functions with the prefix
 ↪  "math."
print(math.pi)

from math import sqrt        # import a single function ...
print(sqrt(2))               # ... and use it without the prefix

import numpy as np           # import a module under a shorter alias
print(np.cos(np.pi))
```

```
4.0
3.141592653589793
1.4142135623730951
-1.0
```

> **Exploring a module**
>
> To find out what a module offers, read its documentation (e.g. the `math` module or NumPy), or type `math.` in Jupyter and press **Tab** to see the available names.

8 Input with `input()`

- Signature: `input(prompt='')`. The `prompt` is displayed, and Python waits until the user types something and presses **Enter**.
- Typical use: `s = input("Question? ")`.
- `input()` **always returns a string**, even if the user types a number. Therefore, we usually need a conversion function to obtain the number we want, e.g. `my_input = int(s)` or `my_input = float(s)`.

> **Interactive cells**
>
> Cells that call `input()` wait for you to type something. They are shown, but not executed, in the HTML version of these notes. Run them yourself in Jupyter!

```
name = input("What is your name? ")
age = input("How old are you? ")    # age holds a string, e.g. "21"
age = int(age)                      # convert the string to an integer
print(f"Hello {name}! Next year you will be {age + 1}.")
```

8.1 Example: the roots of a quadratic equation

We ask the user for the three coefficients of the quadratic equation $c_2x^2 + c_1x + c_0 = 0$ and then compute its two roots,

$$x_{1,2} = \frac{-c_1 \pm \sqrt{\Delta}}{2c_2}, \qquad \text{where } \Delta = c_1^2 - 4c_2c_0$$

is the **discriminant**.

```
from math import sqrt

c0 = float(input("Coefficient of x^0: "))  # input() returns a string, so
 ↪  convert it
c1 = float(input("Coefficient of x^1: "))
c2 = float(input("Coefficient of x^2: "))

discriminant = c1**2 - 4*c2*c0   # value**2 means value to the power of 2
denominator = 2*c2
sqrt_discriminant = sqrt(discriminant)

root_1 = (-c1 + sqrt_discriminant) / denominator
root_2 = (-c1 - sqrt_discriminant) / denominator
print("The two roots are:\n r1 = {0}\n r2 = {1}".format(root_1, root_2))
```

A sample run for the equation $x^2 - 3x + 2 = 0$ looks like this:

```
Coefficient of x^0: 2
Coefficient of x^1: -3
Coefficient of x^2: 1
The two roots are:
 r1 = 2.0
 r2 = 1.0
```

⚠ What if $\Delta < 0$?

If the discriminant is negative, the equation has no real roots, and `sqrt()` stops the program with `ValueError: math domain error`. To deal with this case, the program has to make a *decision* — exactly what the `if` statement does. We will revisit this example in Lecture 2.

Summary

- A **literal** is a fixed value in the code; a **variable** is a name that refers to an object in memory. Variables are created by assignment, updated by reassignment and removed with `del`.
- Follow the naming rules, choose meaningful names, use snake case, and never shadow built-in names such as `sum` or `max`.
- The basic data types are `int`, `float`, `complex`, `bool`, `str` and `NoneType`. Use `type()` to inspect a value and `int()`, `float()`, `str()` to convert between types.
- Operators come in families — arithmetic, comparison, logical, membership, identity and assignment — and follow precedence and associativity rules. When in doubt, use brackets.
- Strings are immutable sequences of characters with many useful methods. Use `print()` with `str.format()` or, preferably, f-strings for tidy output.
- Use `import` to access modules such as `math` and `numpy`.
- `input()` always returns a string; convert it before doing arithmetic.

Exercises

Exercise 1: Just a start

Create two variables `x` and `y`, then calculate $\sqrt{x^2 + y^2}$ using the square-root function from either `math` or `numpy`. Store the result in a variable called `result` and display it with `print()`.

Exercise 2: NumPy and its documentation

Import the library NumPy with the statement

```
import numpy as np
```

then use `np.linspace()` to generate a vector `x` of 20 evenly spaced values from 0 to 2π. Click on numpy.linspace to see its syntax and some examples. Use `np.pi` to obtain the value of π.

> Hint
>
> If you do not understand the explanation of the syntax, go straight to the examples. Once you know more Python, the syntax documentation will make much more sense.

Exercise 3: Mathematical functions

With the help of the `numpy` module where necessary, compute the following expressions:

1. e^2

2. $\sqrt{8}$
3. $\ln(e^{2\pi})$
4. $2^3 + 5^4 + 7$
5. $\cos(2\pi/3)$
6. $\tan(30^\circ)$
7. $\arctan(\sqrt{2})$

N.B. NumPy has two versions of the arctangent. Read the documentation to decide which one you should use here.

> **Hint**
>
> Search the web for *numpy* followed by the name of the function; for example, for e^2 and $\ln(e^{2\pi})$, search for *numpy exponential* and *numpy logarithm*. Also check whether NumPy's trigonometric functions expect angles in degrees or in radians.

Exercise 4: Strings and their operations

Create two strings and store them in two variables `string_1` and `string_2`. Then try the following statements and print the results, in order to understand how the operators work on strings:

1. `string_1 + string_2`
2. `3 * string_1`
3. `string_1 - string_2`

You should receive an error message (a **traceback**) for the third statement, because subtraction is not defined for strings.

The string is a built-in data type with rich documentation and many applications. For example, `my_string.isnumeric()` checks whether `my_string` contains only digits: `'12345'.isnumeric()` returns `True`, while `'a1b2'.isnumeric()` returns `False`.

Finally, after running the statement `import string as st`, try the following and examine the outputs:

- `st.ascii_letters`
- `st.ascii_lowercase`
- `st.ascii_uppercase`
- `st.digits`
- `st.hexdigits`
- `st.octdigits`
- `st.punctuation`
- `st.printable`
- `st.whitespace`

Exercise 5: Fancy output with string formatting

Print the following lines in the output area:

```
4  x 5  = 20
10 x 20 = 200
10 x 5  = 50
4  x 20 = 80
```

All the equals signs `=` and all the letters `x` must be aligned. **Compute the products with real multiplications; do not simply print the text above.** Use string formatting with alignment, for example `print("{0:<n1} x {1:<n2}".format(value1, value2), end=some_string)`, where `n1`, `n2`, `value1` and `value2` are actual numbers, `some_string` is an actual string, and the less-than sign `<` means *aligned to the left*.

Exercise 6: Projectile motion

A body is launched into the air at an angle $\theta = 25^\circ$ (measured from the ground) with an initial velocity of $v_0 = 7.5$ m/s. Assume an acceleration due to gravity of $g = 9.81\,\mathrm{m/s^2}$.

Calculate the following:

1. The initial horizontal velocity $v_{x,0}$ and the initial vertical velocity $v_{y,0}$. Do NumPy's trigonometric functions, e.g. `np.sin()`, work with angles in degrees or in radians? (Use a web search if necessary.) Can you find an easy way to convert between degrees and radians with NumPy?
2. The total time T that the body spends in the air. Remember that the body spends as much time going up as coming down, so $v_{y,0} = gT/2$.
3. The altitude of the body at $t_1 = 200$ ms, $t_2 = 400$ ms and $t_3 = T$. The relevant formula is $h = v_{y,0}\,t - \frac{1}{2}gt^2$; you only have to change the time and re-evaluate it.
4. The distance from the launch position at the times $t_1 = 200$ ms, $t_2 = 400$ ms and $t_3 = T$, according to $d = \sqrt{x^2 + y^2}$, where $x = v_{x,0}\,t$ and $y = v_{y,0}\,t - gt^2/2$.
5. What is the maximum altitude? At what distance from the launch position does the body hit the ground?

Print in style: always print each result together with its meaning and a fixed number of decimal places. For example, if $v_{x,0} = 4/3$, print it as `v0,x = 1.333`. This is easily done with

```
v0x = 4/3
# or, with an f-string: print(f"v0,x = {v0x:.3f}")
print("v0,x = {0:.3f}".format(v0x))
```

Further reading

- The official Python tutorial: An Informal Introduction to Python and Input and Output.
- PEP 8 — Style Guide for Python Code.
- Format Specification Mini-Language, the full reference for `{:...}` specifications.